

APP DEVELOPMENT

ASSIGNMENT NO:01(THEORY)

SUBMITTED TO:

SIR UZAIR

SUBMITTED BY:

NOOR-US-SABA

242201097

1. Introduction and Objectives

The objective of this assignment is to study the mechanism of integrating Machine Learning (ML) and Deep Learning (DL) models into Android applications using TensorFlow Lite (TFLite). This project combines conceptual analysis with practical implementation by engineering a mini application named 'EcoSort AI', which performs real-time, on-device waste classification.

2. Conceptual Study: Embedding ML/DL Models in Mobile Applications

2.1 Meaning and Importance

Embedding ML/DL models into mobile applications refers to running inference locally on the device hardware rather than transmitting data to remote cloud servers. In modern intelligent systems—such as real-time object detection, text auto-completion, and waste sorting—on-device inference enables instant user feedback.

2.2 Benefits and Challenges of On-Device Inference

- **Low Latency:**Eliminates network round-trip time, achieving sub-second response times.
- **Offline Functionality:**Functions reliably without an active cellular or Wi-Fi internet connection.
- **Data Privacy:**Sensitive user photos and camera streams remain strictly on the local storage.
- **Hardware Constraints:**Mobile devices face strict limits regarding RAM, battery life, and thermal throttling.

3. TensorFlow Lite (TFLite) Framework Analysis

3.1 Definition and Standard TensorFlow Comparison

TensorFlow Lite is an open-source, lightweight cross-platform framework optimized for executing machine learning models on resource-constrained mobile and edge devices. Unlike standard TensorFlow, TFLite utilizes FlatBuffers serialization, weight quantization, and reduced operator sets to minimize executable binary size and memory usage.

3.2 Step-by-Step TFLite Android Integration Pipeline

- **1. Model Conversion:**Convert trained SavedModel/Keras structures to serialized .tflite format.
- **2. Asset Deployment:**Place model file in app/src/main/assets alongside labels.txt.
- **3. Interpreter Setup:**Initialize org.tensorflow.lite.Interpreter to load mapped ByteBuffer.

- **4. Input Pre-processing:**Pre-process input images (224x224 RGB bitmap) into normalized float vectors.
- **5. Inference Execution:**Execute interpreter.run(byteBuffer, outputArray) for Softmax prediction.

4. Analysis of Reference GitHub Repository

The provided reference repository (NumberGuessingAppTflite) demonstrates fundamental TFLite interpreter initialization and raw numeric array prediction. The structure includes asset loading via AssetFileDescriptor, converting user inputs into flat buffers, and displaying output logits on the user interface.

Technical Domain	Reference Repository (NumberGuessing)	Proposed Application (EcoSort AI)
Model Type	Digit recognition / Scalar numeric model	Multi-class waste vision classifier (224x224 RGB)
Data Input	Touch canvas / Direct numerical matrix	Camera API capture and MediaStore Gallery Picker
Navigation & UI	Single activity view	Multi-screen lifecycle (Splash -> Login -> Main)

5. EcoSort AI: Proposed Application & Architecture

5.1 Core Purpose and Technologies

EcoSort AI aims to automate waste material classification (Plastic, Organic, Paper, Metal) directly on Android smartphones. Built using Java, Android SDK, TensorFlow Lite Runtime, ActivityResultContracts for secure storage access, and custom splash/authentication views.

5.2 Navigation Lifecycle

- **SplashActivity:**Displays brand icon and transitions automatically via Handler.
- **LoginActivity:**Presents credential input fields and controls application navigation flow.
- **MainActivity:**Houses camera hardware intent and modern gallery content picker.

6. Experimental Testing and Results

System verification confirmed successful model loading and sub-50ms inference times. Testing highlighted that low confidence values (~38%) occur under cluttered backgrounds, which is mitigated by expanding dataset training variations across diverse environments.

7. Conclusion and References

EcoSort AI successfully demonstrates edge-based deep learning integration in Android. The implementation meets all assignment parameters by providing low-latency classification, robust permission handling, and clean modular code.

- **Reference Repository:** https://github.com/Uzair390-Del/NumberGuessingAppTflite.git
- **Framework Docs:** Google TensorFlow Lite Android Documentation.